

00 — Implementation Plan: DVL-Dropout Velocity GP

What you'll learn: the concrete first build, end to end.

Prereqs: references/04 (why);

this doc is the what/how.

Why it matters here: this is the spec for the coding round (separately approved).

Goal

Add a Gaussian Process that models the **DVL body-frame velocity over time** and uses it to **denoise, adaptively weight (R_k), reject outliers from, and bridge dropouts in** the DVL stream feeding the existing error-state EKF — with **zero changes to the EKF core**.

Confirmed posture: **demo-first, accuracy over optimization now; eventual edge**; use a GP library

(GPyTorch). The module is built behind a `predict()/ingest()` interface so the GPyTorch engine can

later be swapped for the $O(n)$ state-space form (references/05, 04-edge-and-spde-roadmap).

Why it needs no EKF-core changes (the key seam)

The existing EKF already accepts a **per-measurement covariance** and already does outlier gating:

- `CorrectedDvlMeasurement` carries `velocity_body_m_s` **and** `covariance_body` (`dvl_correction/src/dvl_imu_kalman.py:38-50`).
- `update_corrected_dvl_velocity` builds R from that covariance via `_velocity_measurement_covariance(...)` (`dvl_imu_kalman.py:213-230`).
- `_apply_measurement_update` already rejects gross outliers with a **Mahalanobis gate** (`dvl_imu_kalman.py:277-300`).

So the GP injects results purely by constructing a

``CorrectedDvlMeasurement(velocity_body_m_s=..., covariance_body=...)`. Adaptive  $R_k$  = "set `covariance_body`` to the GP predictive covariance." See

01-codebase-architecture-and-seams for exact anchors.

Components to build

1. ``dvl_correction/src/gp_velocity.py`` — `GpVelocityConfig` + `GpDvlVelocityModel` with

`ingest / fit / predict / is_outlier`. Multitask (Matérn-3/2 + RQ + RBF) kernel over time → 3-D

velocity, sliding window, periodic refit, dropout-bridge logic. Full design in 02-gp-velocity-module-design.

2. **Pipeline wiring** in `dvl_correction/src/synchronization.py` — an optional `gp_velocity`

component (parallel to the existing `dvl_track`), doing denoise + adaptive- R on accepted DVL and

injecting bridge measurements during silence. Anchors in 01-codebase-architecture-and-seams.

3. Config + exports — a GP block in `dvl_correction/src/config.py` build_navigation_pipeline

(off by default), and exports from `dvl_correction/src/__init__.py`.

4. Dependency — optional extra in `dvl_correction/pyproject.toml`:
[project.optional-dependencies] gp = ["gpytorch>=1.11", "torch>=2.1"];
`gp_velocity.py` imports
`torch/gpytorch lazily` so the core stays numpy-only.

5. Benchmark + demo — `examples/gp_dropout_benchmark.py` (with/without-GP RMSE) and optional

`examples/gp_demo_ws.py` (live frontend). See 03-benchmark-and-frontend-demo.

6. Tests — `dvl_correction/tests/test_gp_velocity.py` (unit) + a pipeline integration test.

The three usage modes (recap)

Mode	Trigger	Action	Effect
Denoise + adaptive 'Rk'	a valid DVL sample arrives	replace velocity with GP mean; set 'covariance_body' = GP cov	cleaner, correctly-weighted updates
Outlier rejection	incoming sample far from GP band	drop before fusion (EKF gate is backstop)	robustness
Dropout bridge	DVL silent > 'bridge_after_s'	inject 'CorrectedDvlMeasurement' from GP prediction, cov grows with gap	graceful degradation to IMU

Build order (suggested)

1. `gp_velocity.py` standalone + unit tests (no pipeline yet) — verify mean/cov, variance growth, outlier flag.
2. Synthetic benchmark harness — generate truth + DVL with dropouts/outliers.
3. Pipeline wiring (denoise + adaptive R first; bridge second), off by default.
4. Run benchmark with/without GP → record RMSE + plot.
5. (Stretch) WebSocket bridge to the React viewer.
6. (Later round) state-space edge port; (future) SPDE current field.

Dependencies & risk

- **New deps:** gpytorch, torch as an **opt-in extra** only. Core unaffected; existing numpy-only tests keep passing (GP off by default).
- **Risk:** torch is heavy and **not** edge-deployable — accepted for the demo; the edge port (04-edge-and-spde-roadmap) removes it. Lighter fallback: `sklearn.gaussian_process` per-axis if torch is unwanted.
- **Risk:** over-smoothing / over-confident bridging — mitigated by Matérn-3/2, learned lengthscale, `max_bridge_s`, and benchmark validation (references/04 §7).

Verification (summary; full detail in doc 03)

1. **Unit:** finite 3-vector mean + SPD 3×3 cov; predictive variance grows with extrapolation; `is_outlier` flags an injected spike; passthrough below `min_samples`.
2. **Integration:** GP-enabled pipeline output shapes identical to baseline; never raises;

`gp_bridge_updates > 0` during a simulated dropout.

3. Benchmark: GP-enabled position RMSE measurably below baseline across dropout windows.

4. Regression: `python3 -m unittest discover -s dvl_correction/tests` stays green (GP off by default).

5. Stretch: `WS bridge + npm run dev` shows a tight covariance ellipse through a dropout.

Key takeaways

- One module + thin pipeline wiring; **EKF untouched** thanks to the `covariance_body` seam.
- GPyTorch now for accuracy; identical interface enables the $O(n)$ edge swap later.
- Validated by a synthetic dropout benchmark and (optionally) the existing covariance-ellipse frontend.

Further reading

- 01 — Codebase architecture and seams
- 02 — `gp_velocity` module design